

Министерство образования Республики Беларусь
г. Минск
Государственное учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»

Лабораторная работа № 2
«Операторы цикла»
по дисциплине «Основы алгоритмизации и программирования»
Вариант 5
Учебная группа 658304

Выполнил: Дерновский А. П.
Проверил: Селезнев А. И.

Минск 2026

Операторы цикла

Лабораторная работа № 2

Дерновский А. П.

Основы алгоритмизации и программирования, группа 658304

вариант 5 · Минск, 2026

ЦЕЛЬ РАБОТЫ

Научиться разрабатывать циклические алгоритмы и писать код на языке Си по составленному алгоритму. Лабораторная работа включает в себя 3 задачи для выполнения. Задачи решаются последовательно.

1 Задание

Описание задания. Население города ежегодно увеличивается на $1/n$ наличного состава жителей, где n — натуральное число. Через сколько лет население города утроится?

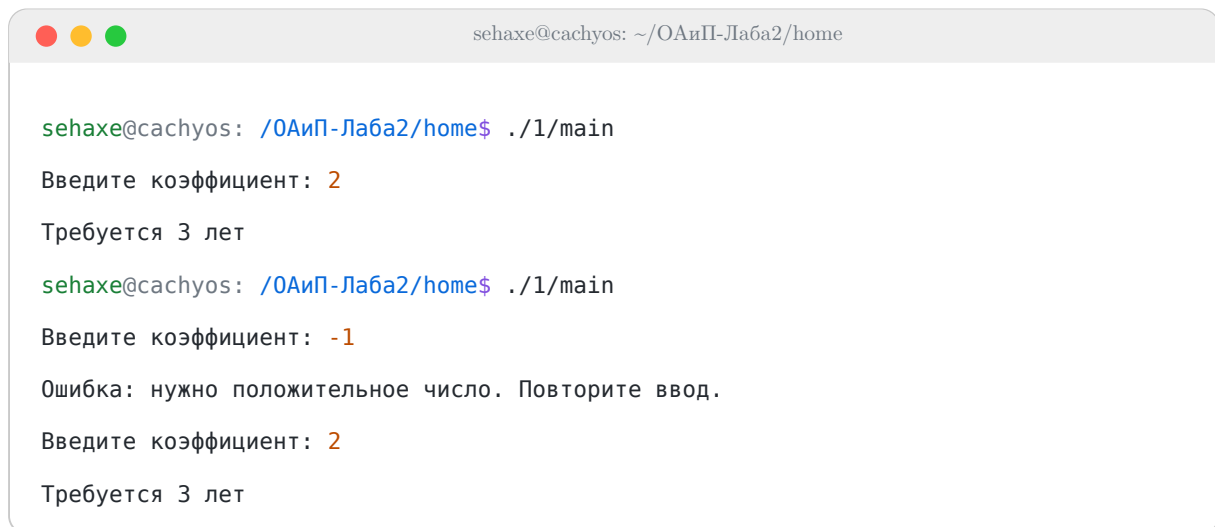
Ход решения. Коэффициент n вводится в цикле `while` с флагом `ok`: флаг заранее сброшен, поэтому тело выполняется минимум один раз; значение должно успешно прочитаться `scanf` и быть положительным — итог проверки запоминается во флаге, а при неверном вводе буфер вычерпывается до конца строки и запрос повторяется. Годовой множитель роста населения равен $1 + 1/n$, поэтому программа хранит текущее население и умножает его на этот множитель в цикле `while`, пока население меньше константы `TARGET` — утроения; счётчик лет `y` растёт вместе с населением. Библиотечное возведение в степень не используется: `math.h` в работе разрешён только для корней и тригонометрии, поэтому рост населения считается умножением в цикле.

```

1  #include <stdio.h>
2
3  //Задача 1. Вариант 5
4  //Население города ежегодно увеличивается на 1/n наличного состава жителей,
5  //где n - натуральное число. Через сколько лет население города утроится?
6
7  #define TARGET 3 //во сколько раз должно вырасти население
8
9  int main(void) {
10     float n;
11     unsigned int y;
12     double population;
13     int c;
14     int ok = 0;
15
16     while (!ok) {
17         printf("Введите коэффициент: ");
18         ok = (scanf("%f", &n) == 1) && (n > 0);
19
20         if (!ok){
21             while ((c = getchar()) != '\n' && c != EOF); //вычерпать плохой ввод
22             printf("Ошибка: нужно положительное число. Повторите ввод.\n");
23         }
24     }
25
26     y = 0;
27     population = 1;
28
29     while (population < TARGET){
30         population = population * (1 + 1/n); //за год население умножается на (1 + 1/n)
31         y++;
32     }
33
34     printf("Требуется %u лет\n", y);
35
36     return 0;
37 }
38

```

Листинг 1. Программа к заданию № 1



```

sehaxe@cachyos: ~/ОАиП-Лаба2/home

sehaxe@cachyos: /ОАиП-Лаба2/home$ ./1/main
Введите коэффициент: 2
Требуется 3 лет

sehaxe@cachyos: /ОАиП-Лаба2/home$ ./1/main
Введите коэффициент: -1
Ошибка: нужно положительное число. Повторите ввод.
Введите коэффициент: 2
Требуется 3 лет

```

Рис. 1. Результаты выполнения задания № 1

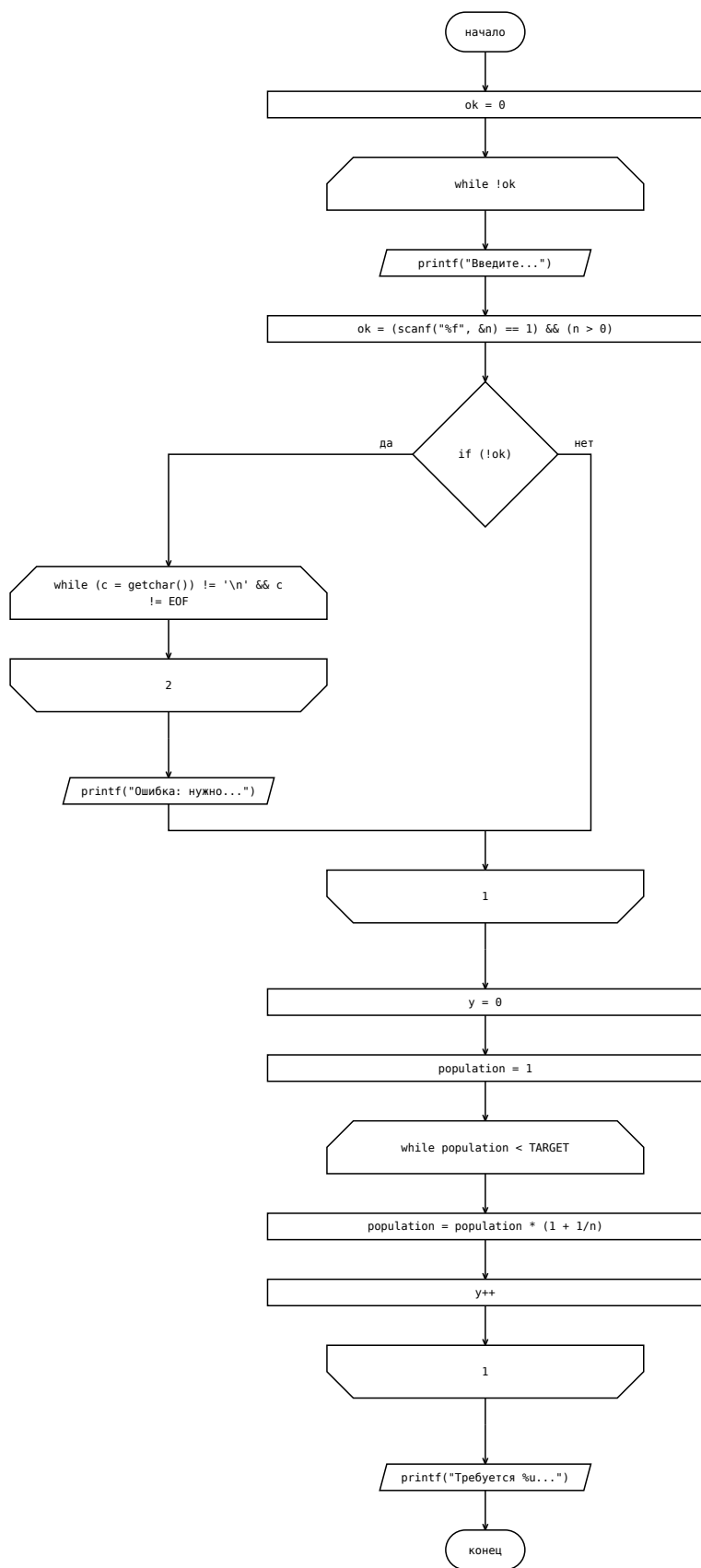


Рис. 2. Блок-схема программы к заданию № 1

2 Задание

Описание задания. Вычислить число Π по формуле Грегори, взяв 500 членов ряда:
 $\Pi/4 = 1 - 1/3 + 1/5 - 1/7 + \dots$

Ход решения. Ряд суммируется циклом `for` из `TERMS` итераций. Знак очередного слагаемого хранится в переменной `sign` и после каждого шага меняется на противоположный умножением на -1 , а знаменатель `denominator` каждый раз увеличивается на `STEP`; частное знак/знаменатель копится в `sum`. Частичная сумма равна $\Pi/4$, поэтому после цикла она умножается на `FACTOR`, и полученное приближение числа Π выводится на экран. Все числовые константы вычислений — 500, 2 и 4 — вынесены в именованные макросы: «голые» цифры в расчётах запрещены условием лабораторной.

```
1  #include <stdio.h>
2
3  //Задача 2. Вариант 5
4  //Вычислить число Пи по формуле Грегори, взяв 500 членов ряда:
5  //П/4 = 1 - 1/3 + 1/5 - 1/7 + ...
6
7  #define TERMS 500 //сколько членов ряда суммируем
8  #define STEP 2    //шаг знаменателя: нечётные числа 1, 3, 5, ...
9  #define FACTOR 4  //частичная сумма равна П/4, значит Пи = сумма * 4
10
11 int main(void) {
12     double sum = 0;
13     double denominator = 1;
14     double sign = 1;
15
16     for (int i = 0; i < TERMS; i++){
17         sum = sum + sign / denominator;
18
19         denominator = denominator + STEP;
20         sign = sign * (-1);
21     }
22
23     double pi = sum * FACTOR;
24
25     printf("Число Пи равно: %f\n", pi);
26
27     return 0;
28 }
29
```

Листинг 2. Программа к заданию № 2

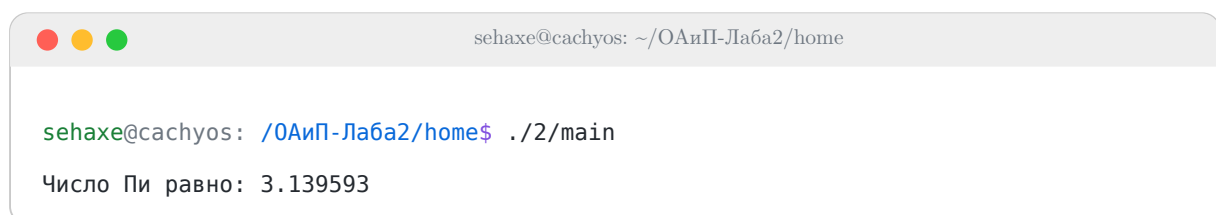


Рис. 3. Результаты выполнения задания № 2

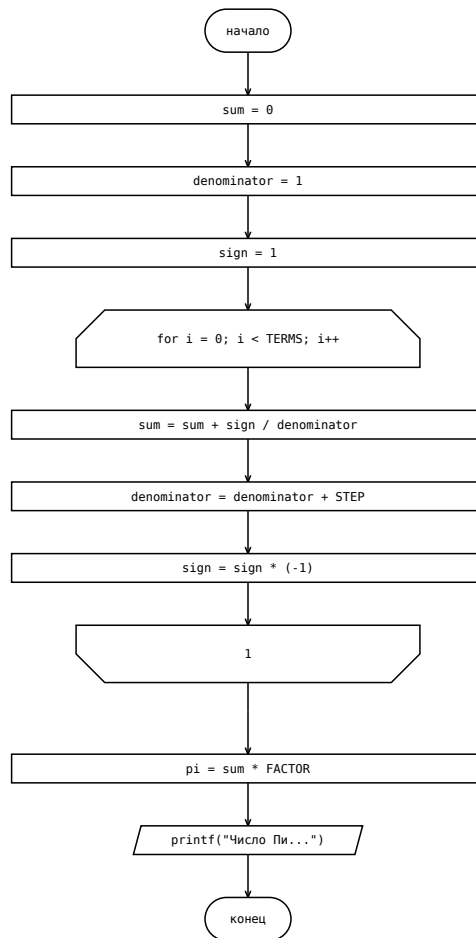


Рис. 4. Блок-схема программы к заданию № 2

3 Задание

Описание задания. Даны целые положительные числа A и B . Найти их наибольший общий делитель (НОД), используя алгоритм Евклида.

Ход решения. Ввод проверяется в цикле `while` с флагом `ok`: функция `scanf` должна успешно прочитать два числа, и оба должны быть положительными; иначе буфер вычерпывается до конца строки и запрос повторяется, пока флаг не окажется истинным. Числа читаются во временные знаковые `ta` и `tb` со спецификатором `%d` — беззнаковый `scanf` принял бы `-3` за гигантское положительное и проверка его пропустила бы; корректные значения переписываются в беззнаковые `a` и `b`. Далее работает алгоритм Евклида — цикл `while` продолжается, пока `b` не ноль; на каждом шаге пара (a, b) заменяется на $(b, a \% b)$, для чего остаток от деления сначала сохраняется во временной переменной `c`. Когда `b` обнуляется, в `a` остаётся наибольший общий делитель исходных чисел.

```

1 #include <stdio.h>
2
3 //Задача 3. Вариант 5
4 //Даны целые положительные числа А и В. Найти их наибольший общий
5 //делитель (НОД), используя алгоритм Евклида.
6
7 int main(void) {
8     unsigned int a, b, c;
9     int ta, tb; //ввод читается в знаковые: %i принял бы -3 за 4294967293
10    int d;       //символ для вычерпывания буфера ввода
11    int ok = 0;
12
13    while (!ok) {
14        printf("Введите числа а и b: ");
15
16        ok = (scanf("%d %d", &ta, &tb) == 2) && (ta > 0) && (tb > 0);
17
18        if (!ok) {
19            while ((d = getchar()) != '\n' && d != EOF); //вычерпать плохой ввод
20            printf("Ошибка: нужны два положительных числа. Повторите ввод.\n");
21        }
22    }
23
24    a = ta;
25    b = tb;
26
27    while (b != 0){
28        c = a % b;
29        a = b;
30        b = c;
31    }
32
33    printf("НОД: %u\n", a);
34
35    return 0;
36 }
37

```

Листинг 3. Программа к заданию № 3

```

sehaxe@cachyos: ~/ОАиП-Лаба2/home

sehaxe@cachyos: /ОАиП-Лаба2/home$ ./3/main
Введите числа а и b: 48 36
НОД: 12

sehaxe@cachyos: /ОАиП-Лаба2/home$ ./3/main
Введите числа а и b: 0 5
Ошибка: нужны два положительных числа. Повторите ввод.
Введите числа а и b: 48 36
НОД: 12

```

Рис. 5. Результаты выполнения задания № 3

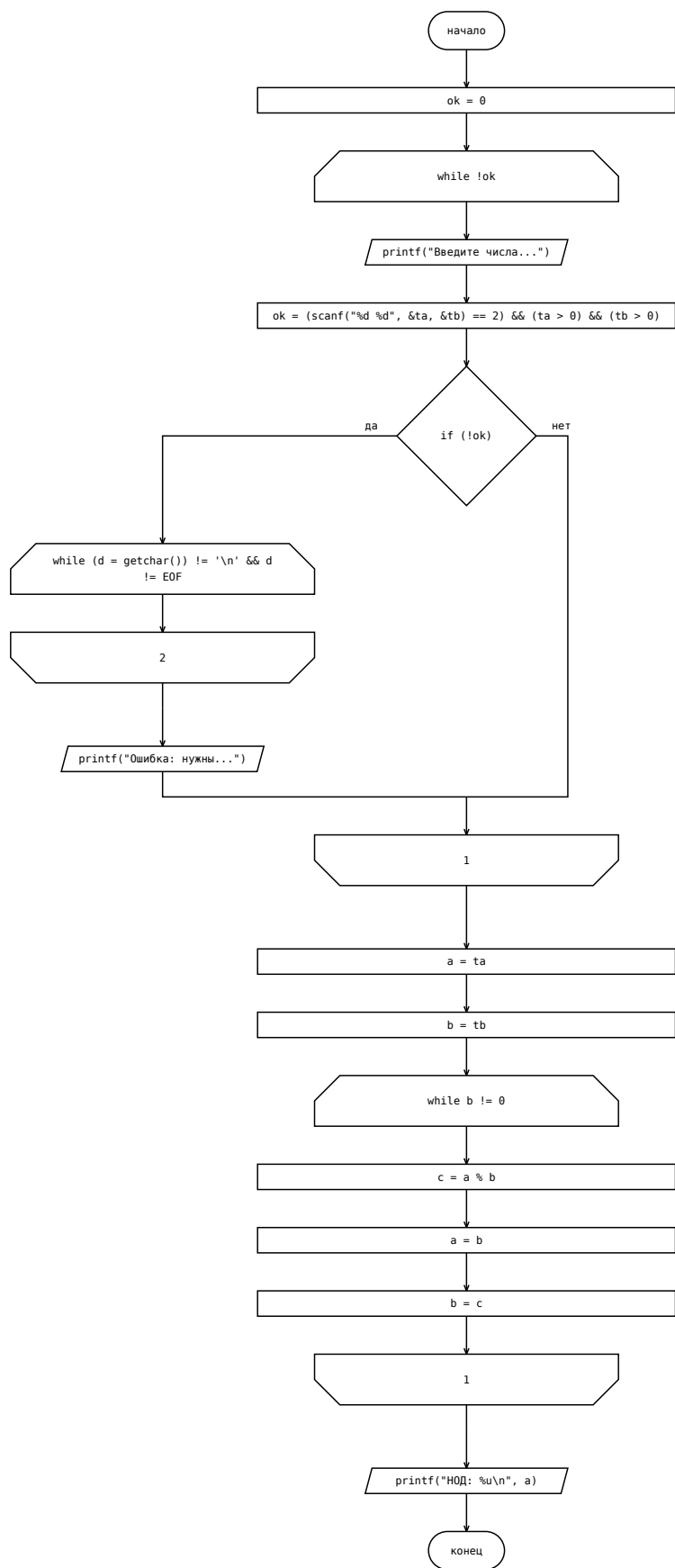


Рис. 6. Блок-схема программы к заданию № 3